Principal Component Analysis (PCA)

Mohammed sharif

Principal Component Analysis (PCA)

This assignment focuses on conducting a Principal Component Analysis (PCA) on a given dataset. The analysis is performed in two parts:

1. First, a manual PCA process is conducted without using built-in functions like `prcomp` to gain a deeper understanding of each step involved in PCA, including standardization, covariance matrix calculation, eigen decomposition, and projection of data onto a new basis.
2. In the second part, the `prcomp` function is used to compare the results, followed by advanced visualization techniques to provide a more comprehensive view of the data. Additional dimensionality reduction techniques, such as clustering using K-Means, are included to further strengthen the analysis.

Load necessary libraries

```
if (!require("readr")) {
    install.packages("readr")
    library(readr)
}
```

```
## Loading required package: readr
```

```
if (!require("ggplot2")) {
    install.packages("ggplot2")
    library(ggplot2)
}
```

```
## Loading required package: ggplot2
```

```
if (!require("ggbiplot")) {
    devtools::install_github("vqv/ggbiplot")
    library(ggbiplot)
}
```

```
## Loading required package: ggbiplot
```

```
## Loading required package: plyr
```

```
## Loading required package: scales
```

```
##
## Attaching package: 'scales'

## The following object is masked from 'package:readr':
##
##     col_factor

## Loading required package: grid
```

Loading the CSV file and centering the data

```
data <- read.csv('anonymous-data.csv')
head(data)
```

```
##   brand    id  mois  prot   fat  ash sodium carb  cal
## 1     A 14069 27.82 21.43 44.87 5.11   1.77 0.77 4.93
## 2     A 14053 28.49 21.26 43.89 5.34   1.79 1.02 4.84
## 3     A 14025 28.35 19.99 45.78 5.08   1.63 0.80 4.95
## 4     A 14016 30.55 20.15 43.13 4.79   1.61 1.38 4.74
## 5     A 14005 30.49 21.28 41.65 4.82   1.64 1.76 4.67
## 6     A 14075 31.14 20.23 42.31 4.92   1.65 1.40 4.67
```

```
numeric_data = data[,2:dim(data)[2]]
head(numeric_data)
```

Removing non-numeric columns and Standardize the data for computing the covariance matrix

```
##      id  mois  prot   fat  ash sodium carb  cal
## 1 14069 27.82 21.43 44.87 5.11   1.77 0.77 4.93
## 2 14053 28.49 21.26 43.89 5.34   1.79 1.02 4.84
## 3 14025 28.35 19.99 45.78 5.08   1.63 0.80 4.95
## 4 14016 30.55 20.15 43.13 4.79   1.61 1.38 4.74
## 5 14005 30.49 21.28 41.65 4.82   1.64 1.76 4.67
## 6 14075 31.14 20.23 42.31 4.92   1.65 1.40 4.67
```

Data Standardization Explanation

Standardization ensures all variables have equal weight, preventing those with larger scales from dominating the PCA.

```
standardized_data <- scale(numeric_data)
```

Covariance Matrix

The covariance matrix shows how much variables vary together, helping identify directions of maximum variance.

```
cov_matrix <- cov(standardized_data)
print(cov_matrix)
```

```
##                 id        mois        prot         fat         ash      sodium
## id      1.00000000  0.03259505 -0.07648546 -0.00486006 -0.03407156 -0.01515608
## mois    0.03259505  1.00000000  0.36024768 -0.17131821  0.26555552 -0.10227890
## prot   -0.07648546  0.36024768  1.00000000  0.49800167  0.82384370  0.42912952
## fat    -0.00486006 -0.17131821  0.49800167  1.00000000  0.79163396  0.93332522
## ash    -0.03407156  0.26555552  0.82384370  0.79163396  1.00000000  0.80812215
## sodium -0.01515608 -0.10227890  0.42912952  0.93332522  0.80812215  1.00000000
## carb    0.01496614 -0.59180165 -0.85354226 -0.64023817 -0.89898837 -0.62017634
## cal    -0.02120862 -0.76444054  0.07025810  0.76456710  0.32646845  0.67195750
##               carb         cal
## id      0.01496614 -0.02120862
## mois   -0.59180165 -0.76444054
## prot   -0.85354226  0.07025810
## fat    -0.64023817  0.76456710
## ash    -0.89898837  0.32646845
## sodium -0.62017634  0.67195750
## carb    1.00000000 -0.02348458
## cal    -0.02348458  1.00000000
```

Eigen decomposition of the covariance matrix

```
eig <- eigen(cov_matrix)
eigenvalues <- eig$values
eigenvectors <- eig$vectors
```

printing the eiganvalues and its vectors

```
for (i in 1:length(eigenvalues)) {
  cat("\nEigenvalue : ", eigenvalues[i], "\n")
  cat("Corresponding eigenvectors\n ",eigenvectors[,i], "\n")
}
```

```
##
## Eigenvalue :  4.173044
## Corresponding eigenvectors
##   -0.0199403 0.06447855 0.3789511 0.4464674 0.4718138 0.435551 -0.4247558 0.2444942
##
## Eigenvalue :  2.290501
## Corresponding eigenvectors
##   -0.005817631 -0.6283753 -0.2696096 0.2342153 -0.1110651 0.2015218 0.3204165 0.5674075
```

```
##
## Eigenvalue :  1.006715
## Corresponding eigenvectors
##   0.9932501 0.04896352 -0.08224737 0.04523198 0.0003454642 0.04337437 -0.01899236 0.002044511
##
## Eigenvalue :  0.4071149
## Corresponding eigenvectors
##   0.1113317 -0.4177734 0.7405123 -0.1930361 0.0557179 -0.4567407 0.04909033 0.1154244
##
## Eigenvalue :  0.09478774
## Corresponding eigenvectors
##   0.02079846 -0.2225595 -0.00886181 -0.5068695 0.5554312 0.4418652 0.3344201 -0.2783056
##
## Eigenvalue :  0.0274903
## Corresponding eigenvectors
##   -0.01398131 -0.004557391 -0.3897736 0.1755801 0.6684675 -0.6032706 0.006064852 0.07880922
##
## Eigenvalue :  0.0003375441
## Corresponding eigenvectors
##   0.0002583986 0.4464539 -6.964216e-05 -0.5254277 0.0588126 0.003212595 -0.0004324985 0.7218969
##
## Eigenvalue :  9.509064e-06
## Corresponding eigenvectors
##   -9.967067e-05 0.4185456 0.2767338 0.3777325 0.05603683 -0.0005556865 0.7760615 0.01201122
```

Sort eigenvalues and eigenvectors in descending order

```
sorted_indices <- order(eigenvalues, decreasing = TRUE)
sorted_eigenvalues <- eigenvalues[sorted_indices]
sorted_eigenvectors <- eigenvectors[, sorted_indices]
```

printing the sorted eiganvalues and its vectors

```
for (i in 1:length(sorted_eigenvalues)) {
  cat("\nEigenvalue : ", sorted_eigenvalues[i], "\n")
  cat("Corresponding eigenvectors\n ",sorted_eigenvectors[,i], "\n")
}
```

```
##
## Eigenvalue :  4.173044
## Corresponding eigenvectors
##   -0.0199403 0.06447855 0.3789511 0.4464674 0.4718138 0.435551 -0.4247558 0.2444942
##
## Eigenvalue :  2.290501
## Corresponding eigenvectors
##   -0.005817631 -0.6283753 -0.2696096 0.2342153 -0.1110651 0.2015218 0.3204165 0.5674075
##
## Eigenvalue :  1.006715
## Corresponding eigenvectors
##   0.9932501 0.04896352 -0.08224737 0.04523198 0.0003454642 0.04337437 -0.01899236 0.002044511
```

```
##
## Eigenvalue :  0.4071149
## Corresponding eigenvectors
##   0.1113317 -0.4177734 0.7405123 -0.1930361 0.0557179 -0.4567407 0.04909033 0.1154244
##
## Eigenvalue :  0.09478774
## Corresponding eigenvectors
##   0.02079846 -0.2225595 -0.00886181 -0.5068695 0.5554312 0.4418652 0.3344201 -0.2783056
##
## Eigenvalue :  0.0274903
## Corresponding eigenvectors
##   -0.01398131 -0.004557391 -0.3897736 0.1755801 0.6684675 -0.6032706 0.006064852 0.07880922
##
## Eigenvalue :  0.0003375441
## Corresponding eigenvectors
##   0.0002583986 0.4464539 -6.964216e-05 -0.5254277 0.0588126 0.003212595 -0.0004324985 0.7218969
##
## Eigenvalue :  9.509064e-06
## Corresponding eigenvectors
##   -9.967067e-05 0.4185456 0.2767338 0.3777325 0.05603683 -0.0005556865 0.7760615 0.01201122
```

Form the matrix of change of basis

Each column in this matrix represents a principal component, which is a direction in the feature space that explains the maximum variance in the data. The first column corresponds to the first principal component, and so on.

```r
change_of_basis_matrix <- sorted_eigenvectors
head(change_of_basis_matrix)
```

```
##             [,1]         [,2]          [,3]       [,4]        [,5]         [,6]
## [1,] -0.01994030 -0.005817631  0.9932500519  0.1113317  0.02079846 -0.013981307
## [2,]  0.06447855 -0.628375256  0.0489635165 -0.4177734 -0.22255948 -0.004557391
## [3,]  0.37895107 -0.269609639 -0.0822473744  0.7405123 -0.00886181 -0.389773631
## [4,]  0.44646740  0.234215299  0.0452319845 -0.1930361 -0.50686952  0.175580059
## [5,]  0.47181383 -0.111065055  0.0003454642  0.0557179  0.55543119  0.668467507
## [6,]  0.43555098  0.201521828  0.0433743722 -0.4567407  0.44186519 -0.603270611
##               [,7]          [,8]
## [1,]  2.583986e-04 -9.967067e-05
## [2,]  4.464539e-01  4.185456e-01
## [3,] -6.964216e-05  2.767338e-01
## [4,] -5.254277e-01  3.777325e-01
## [5,]  5.881260e-02  5.603683e-02
## [6,]  3.212595e-03 -5.556865e-04
```

Project the data onto the new basis (new representation in principal component space)

```r
pca_transformed_data <- standardized_data %*% change_of_basis_matrix

# Convert the transformed data to a data frame
pca_transformed_data <- as.data.frame(pca_transformed_data)
```

```r
colnames(pca_transformed_data) <- paste0("PC", 1:ncol(pca_transformed_data))

# Inspect the transformed data
head(pca_transformed_data)
```

```
##        PC1      PC2        PC3        PC4         PC5         PC6           PC7
## 1 5.020614 2.679389 -0.8535710 -0.1388168  0.12406416 -0.27156189  0.002146848
## 2 5.034030 2.529728 -0.8533401 -0.1975070  0.33349078 -0.20356758 -0.003136189
## 3 4.816163 2.673930 -0.8505083 -0.1728519 -0.12639543  0.04810741 -0.005659568
## 4 4.480891 2.285987 -0.8596403 -0.2193752 -0.07394952 -0.16104967 -0.005836865
## 5 4.483324 2.160369 -0.8805290 -0.1027267  0.09682088 -0.30024406 -0.001874878
## 6 4.515901 2.169070 -0.8488869 -0.2739931  0.09505921 -0.18794175 -0.005392061
##             PC8
## 1 -0.0008861239
## 2 -0.0009470118
## 3 -0.0007355269
## 4 -0.0008596368
## 5 -0.0008936227
## 6 -0.0008969281
```

Variance Explained Plot

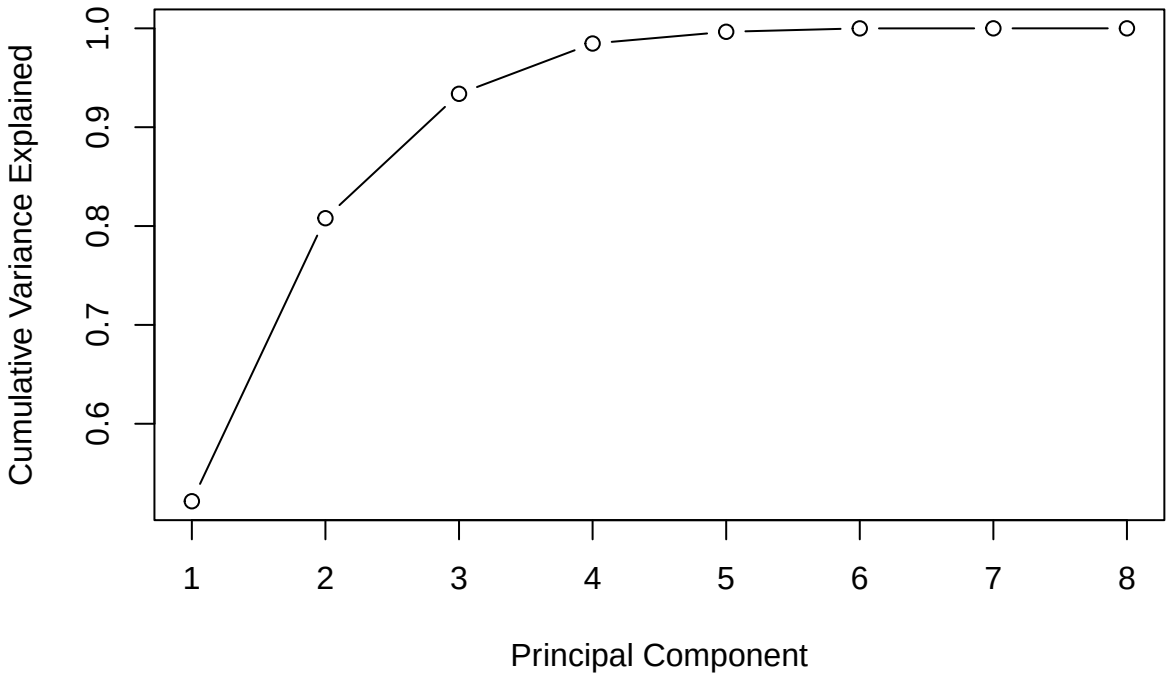
Plot the cumulative variance explained by the principal components

```r
explained_variance <- sorted_eigenvalues / sum(sorted_eigenvalues)
cumulative_variance <- cumsum(explained_variance)
```

Plot cumulative variance to show how many components are needed to explain most of the variance

Interpretation: From the plot, we can observe how many principal components are required to explain a significant portion of the variance (usually around 90%). This allows us to decide how many components to keep for further analysis.

```r
plot(cumulative_variance, type = "b", xlab = "Principal Component", ylab = "Cumulative Variance Explain
```




```
threshold <- 0.9  # Set your threshold for explained variance (90%)
components_needed <- which(cumulative_variance >= threshold)[1]
cat("\nNumber of components required to explain at least 90% of variance: ", components_needed, "\n")
```

```
##
## Number of components required to explain at least 90% of variance:  3
```

Part 2: PCA using prcomp

```
# Perform PCA using prcomp
pca_result <- prcomp(numeric_data, scale. = TRUE)

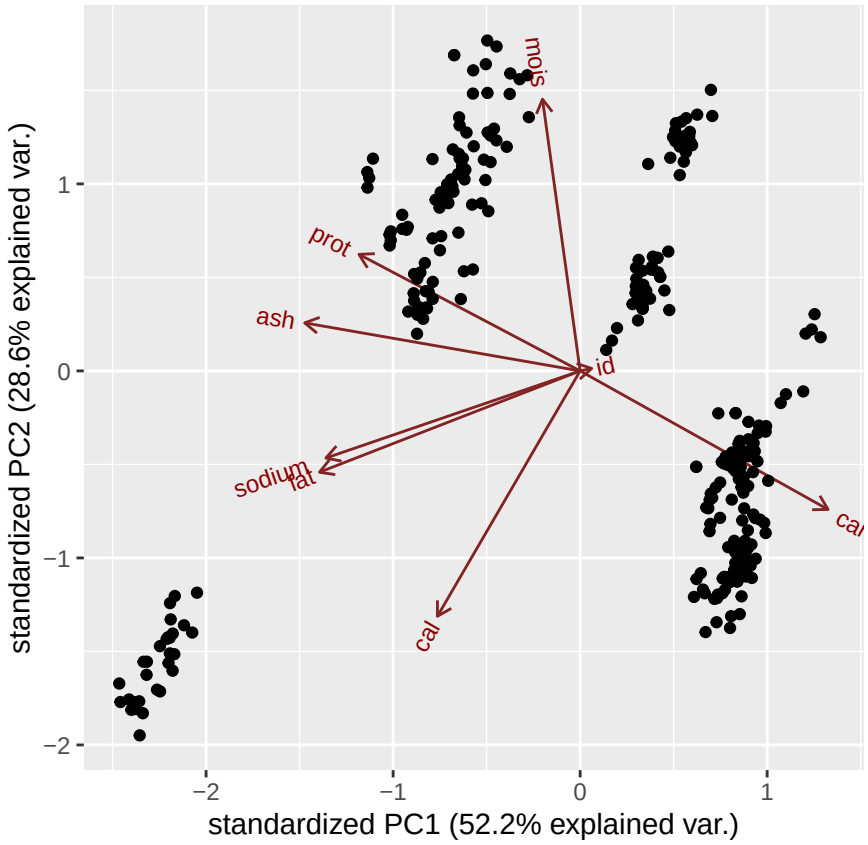
# Summary of the PCA
summary(pca_result)
```

```
## Importance of components:
##                           PC1    PC2    PC3     PC4     PC5     PC6     PC7
## Standard deviation     2.0428 1.5134 1.0034 0.63806 0.30788 0.16580 0.01837
## Proportion of Variance 0.5216 0.2863 0.1258 0.05089 0.01185 0.00344 0.00004
## Cumulative Proportion  0.5216 0.8079 0.9338 0.98467 0.99652 0.99996 1.00000
##                            PC8
## Standard deviation     0.003084
```

```
## Proportion of Variance 0.000000
## Cumulative Proportion  1.000000
```

```
# Load necessary libraries

# Plot the PCA biplot
ggbiplot(pca_result)
```

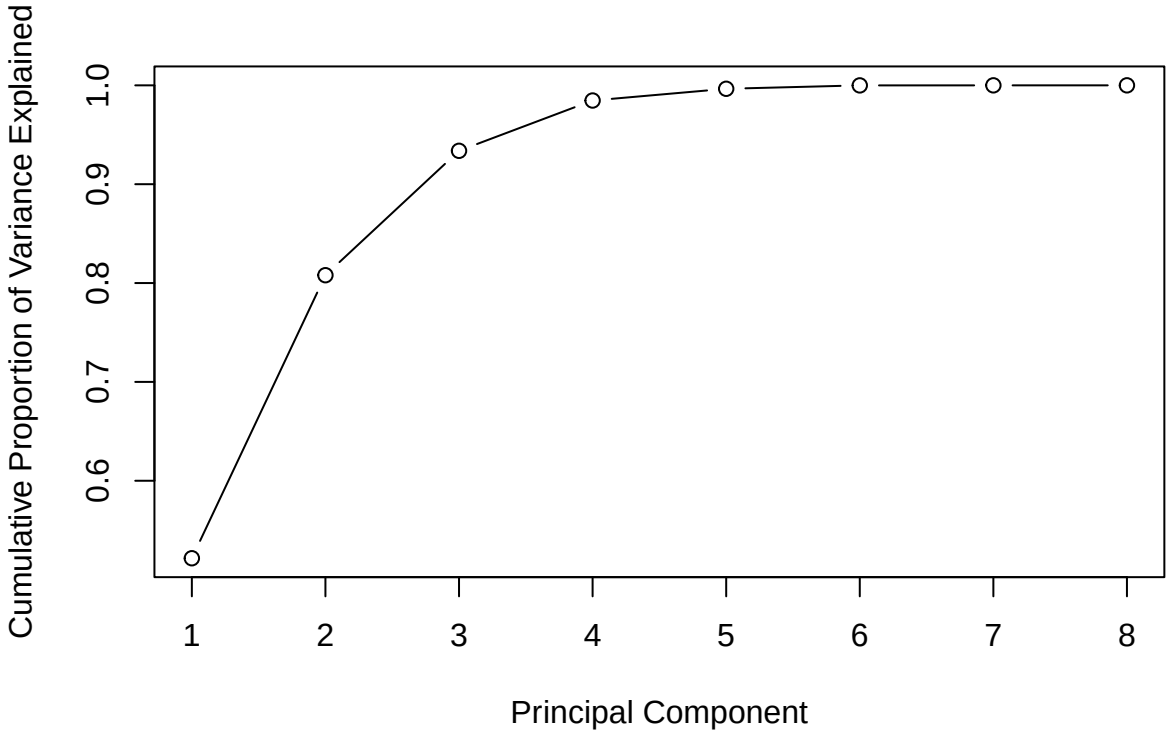


Cumulative variance plot using prcomp results

```
explained_variance_prcomp <- pca_result$sdev^2 / sum(pca_result$sdev^2)

cumulative_variance_prcomp <- cumsum(explained_variance_prcomp)

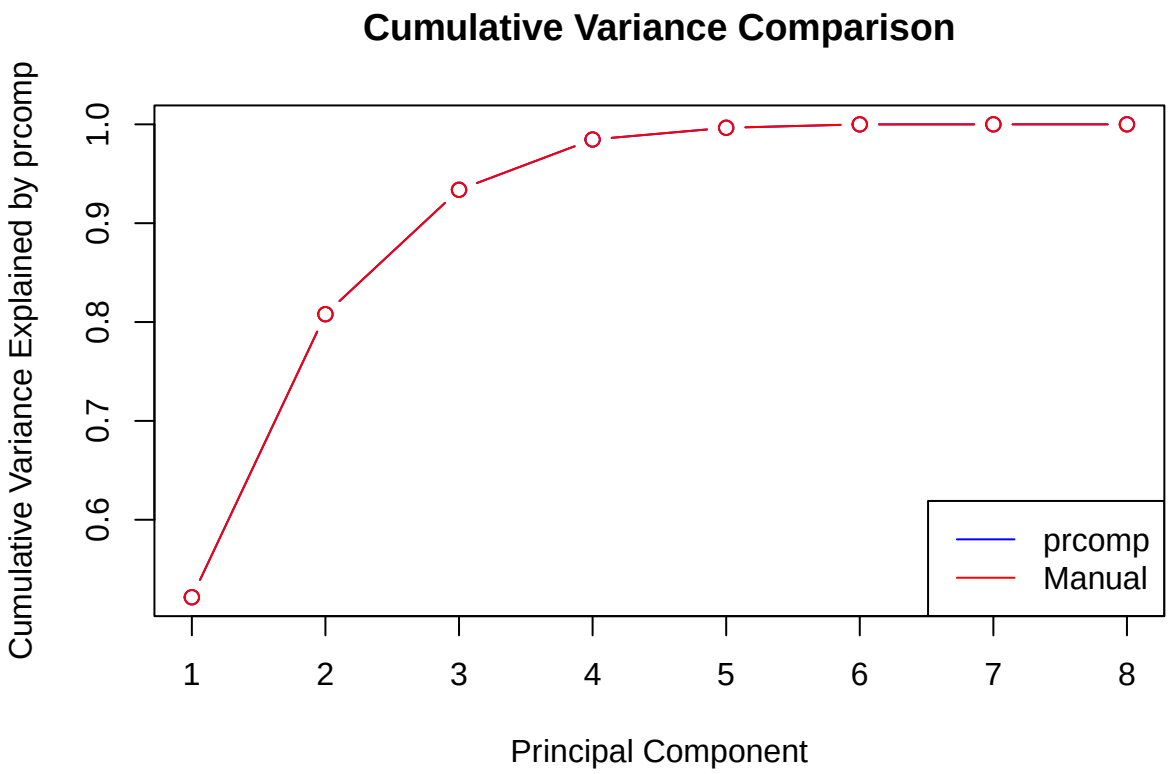
# Plot
plot(cumulative_variance_prcomp, xlab = "Principal Component", ylab = "Cumulative Proportion of Variance
```



Comparison between manual PCA and prcomp.

Interpretation : The red line represents the manually calculated PCA, and the blue line shows the results from prcomp. If the two lines are close, it confirms that the manual PCA is implemented correctly.

```
# Plot the comparison between manual PCA and prcomp results
plot(cumulative_variance_prcomp, type = "b", xlab = "Principal Component", ylab = "Cumulative Variance E
lines(cumulative_variance, type = "b", col = "red")
legend("bottomright", legend = c("prcomp", "Manual"), col = c("blue", "red"), lty = 1)
```

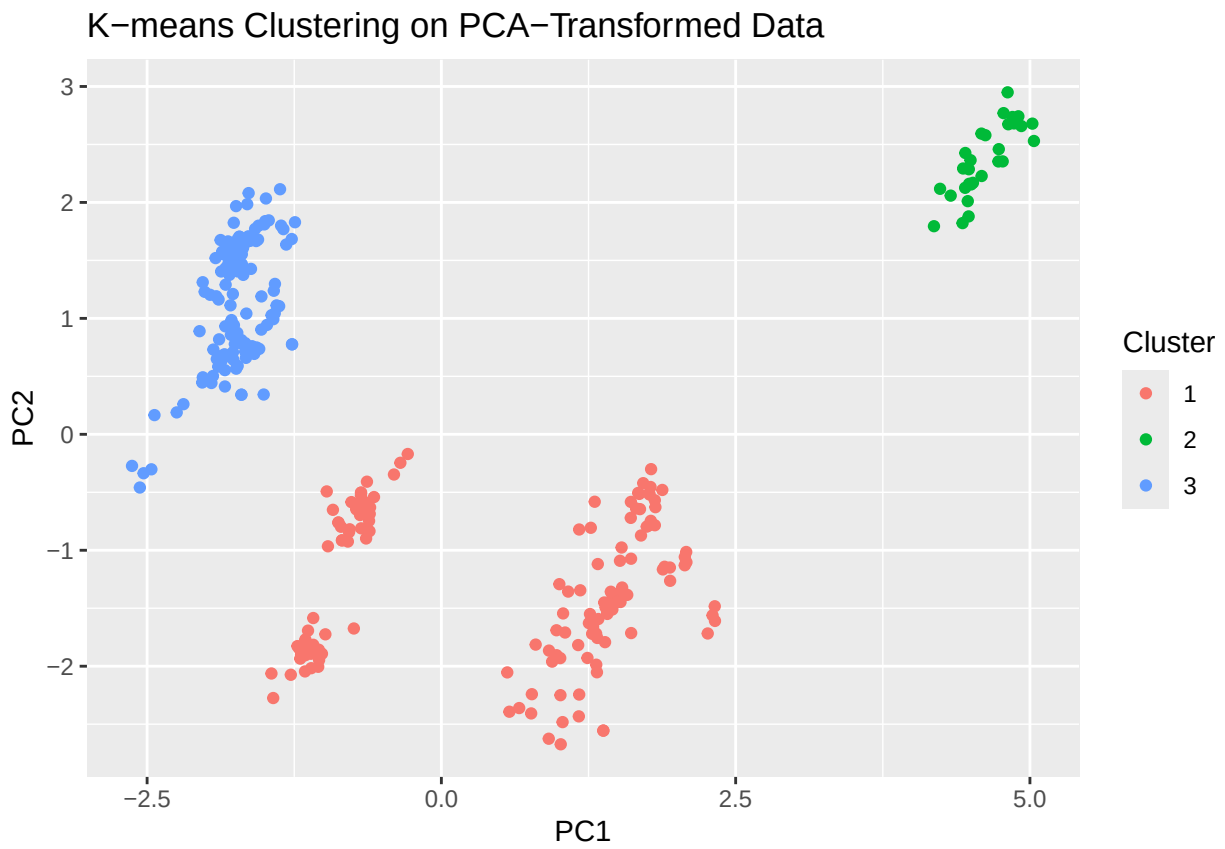


K-Means Clustering on PCA-Transformed Data

set.seed(123) ensures consistent clustering results by fixing the randomness in the K-means algorithm. The K-means clustering is performed on the top principal components that capture the most variance, specifying 3 clusters. The plot visualizes these clusters on the first two principal components (PC1, PC2), with each cluster distinguished by a different color for clearer interpretation.

```
set.seed(123)  # For reproducibility
kmeans_result <- kmeans(pca_transformed_data[, 1:components_needed], centers = 3)  # Assuming 3 cluster

# Plot the clusters
ggplot(pca_transformed_data, aes(x = PC1, y = PC2, color = as.factor(kmeans_result$cluster))) +
  geom_point() +
  labs(title = "K-means Clustering on PCA-Transformed Data", x = "PC1", y = "PC2", color = "Cluster")
```



Conclusion

In this analysis, we performed a Principal Component Analysis (PCA) both manually and using the `prcomp` function to reduce the dimensionality of the dataset and identify key patterns. We observed how the majority of the variance in the data is captured by the first few principal components, allowing us to simplify the data without losing significant information. Additionally, we applied K-means clustering on the PCA-transformed data, revealing distinct groupings within the dataset, and visualized these clusters using the first two principal components. This comprehensive approach not only helped in understanding the underlying structure of the data but also demonstrated the effectiveness of PCA in reducing complexity while retaining essential patterns.